

TP1 - Maël PEROT

1)

La requête SQL qui permet d'obtenir pour chaque auteur le nombre d'articles qu'il a rédigés est la suivante :

```
SELECT aut.personneId, aut.nom, aut.prenom, COUNT(*) AS nb_articles FROM cg_article art
INNER JOIN cg_rediger r ON r.articleid = art.articleid
INNER JOIN cg_auteur aut ON aut.personneid = r.personneid
GROUP BY aut.personneid
```

La procédure stockée qui permet d'obtenir le nombre d'articles qu'un auteur a écrit est :

```
CREATE OR REPLACE FUNCTION nbArticlesRediges (INTEGER)
RETURNS INTEGER
AS $$
DECLARE
nbArticles INTEGER = 0;
BEGIN
SELECT COUNT(*) INTO nbArticles from cg_article art
INNER JOIN cg_rediger r ON r.articleid = art.articleid
INNER JOIN cg_auteur aut ON aut.personneid = r.personneid
WHERE aut.personneid = $1;
RETURN nbArticles;
END;
$$ LANGUAGE plpgsql;
```

La requête SQL qui permet d'obtenir pour chaque auteur (son identifiant, son nom, son prénom) le nombre d'articles qu'il a rédigé en utilisant la procédure stockée écrite est :

```
SELECT personneid, nom, prenom, nbArticlesRediges(personneid) FROM cg_auteur
```

2)

La requête qui permet de modifier la table auteur pour y ajouter le champ « nbart » est :

```
ALTER TABLE cg_auteur
ADD COLUMN nbart INTEGER NOT NULL DEFAULT 0;
```

Pour mettre à jour le champ « nbart » de la table auteur, on fait :

```
UPDATE cg_auteur
SET nbart=(SELECT nbArticlesRediges(personneid));
```

Le mécanisme qu'on va mettre en place est un trigger. On l'écrit de la façon suivante :

```
CREATE OR REPLACE FUNCTION updateNbArt()
RETURNS TRIGGER AS $$
BEGIN
IF TG_OP = 'DELETE' OR TG_OP = 'UPDATE' THEN
UPDATE cg_auteur
SET nbart = nbart-1
WHERE cg_auteur.personneId = OLD.personneId;
END IF;
IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
UPDATE cg_auteur
SET nbart = nbart+1
WHERE cg_auteur.personneId = NEW.personneId;
END IF;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER tg_nbart AFTER DELETE OR INSERT OR UPDATE ON
cg_rediger
EXECUTE PROCEDURE updateNbArt();
```

3)

Le trigger qui à l'insertion d'un participant ou d'un auteur insère automatiquement la personne correspondante est :

```
CREATE OR REPLACE FUNCTION insertPerson()
RETURNS TRIGGER AS $$
BEGIN
INSERT INTO cg_personne (personneid, nom, prenom, email)
VALUES (NEW.personneId, NEW.nom, NEW.prenom, NEW.email);
RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER tg_participantInPersonne AFTER INSERT ON cg_participant
EXECUTE PROCEDURE insertPerson();
CREATE OR REPLACE TRIGGER tg_participantInPersonne AFTER INSERT ON cg_auteur
EXECUTE PROCEDURE insertPerson();
```

Le trigger qui à la modification d'un participant ou d'un auteur modifie ces mêmes informations dans la table personne est:

```
CREATE OR REPLACE FUNCTION replacePerson()
RETURNS TRIGGER AS $$
BEGIN
UPDATE cg_personne
```

```
SET nom = NEW.nom,  
prenom = NEW.prenom,  
email = NEW.email  
WHERE personneid = NEW.personneid;  
RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER tg_participantInPersonne AFTER UPDATE ON cg_participant  
EXECUTE PROCEDURE replacePerson();  
CREATE OR REPLACE TRIGGER tg_participantInPersonne AFTER UPDATE ON cg_auteur  
EXECUTE PROCEDURE replacePerson();
```

Le trigger qui à la suppression d'un participant ou d'un auteur supprime cette personne dans la table personne est:

```
CREATE OR REPLACE FUNCTION deletePerson()  
RETURNS TRIGGER AS $$  
BEGIN  
DELETE FROM cg_personne  
WHERE personneid = OLD.personneid;  
RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER tg_supressParticipant AFTER DELETE ON cg_participant  
EXECUTE PROCEDURE deletePerson();  
CREATE OR REPLACE TRIGGER tg_supressAuteur AFTER DELETE ON cg_auteur  
EXECUTE PROCEDURE deletePerson();
```